

Track-System – Backend Files Map

Last updated: 22 Sep 2026 | Covers everything up to step 2 | What each file is for and how they are wired together

1. Where everything sits

```
Track-System/  
  Frontend/ (React + Vite - already built)  
    src/pages/Login, ForgotPassword, ChangePassword  
  Backend/  
    TrackSystem.Api/  
      TrackSystem.Api.csproj <- list of libraries  
      appsettings.json <- settings (connection string)  
      Program.cs <- the server: setup + endpoints  
    Data/  
      AppDbContext.cs <- the bridge to the database  
    bin/ obj/ (build output - ignored by git)  
    .gitignore (what git must not upload)
```

2. What each file does

File	What it is	What it does	When do I touch it
TrackSystem.Api.csproj	Project definition	Lists the .NET version and every library the project uses. Today: EntityFrameworkCore.SqlServer.	Every time we add a new library
appsettings.json	Settings file (not code)	Holds values the code needs: the connection string (server name + DB name). Later also the JWT key.	When the server name, the DB or a key changes
Data/AppDbContext.cs	The bridge to the DB	Represents the database in C#. Every query passes through it. Will hold one line per table.	Every time we add a table
Program.cs	The server	Two parts: the setup area (registers the DB, later CORS and JWT), then the endpoints (the /api/... addresses).	Every new endpoint or new service
.gitignore	Git rules	Stops build files (bin, obj, node_modules) and secret files from being uploaded to GitHub.	Rarely

3. How the files are connected

```
Browser / React --(HTTP request /api/...)--> Program.cs  
  |  
  | Program.cs reads the connection string from --> appsettings.json  
  | Program.cs uses the classes from -----> Data/AppDbContext.cs  
  | both are possible only because -----> .csproj installed EF Core  
  v  
AppDbContext --(EF Core turns C# into SQL)--> SQL Server, DB FinalProject  
  |  
  v  
rows --> C# objects --> JSON --> back to the browser
```

This file	needs / uses	Why
Program.cs	appsettings.json	Asks for the value named "DefaultConnection". If the name is misspelled in either file, the connection fails.

Program.cs	Data/AppDbContext.cs	Registers it in the setup area, then receives a ready one inside each endpoint.
AppDbContext.cs	EF Core (from the .csproj)	It inherits from DbContext, which only exists because the library is installed.
Everything	TrackSystem.Api.csproj	If the library version does not match the .NET version, nothing builds at all.

4. The addresses that exist today (all inside Program.cs)

Address	Answers	What it proves
/api/health	{"status":"ok"}	The server itself is alive
/api/db-check	{"database":"connected"}	The server reached the FinalProject database

5. Rules for this project

Rule	Why
Services are registered above <code>var app = builder.Build();</code>	After the server is built, nothing can be registered any more.
Server names, DB names, keys and passwords live in appsettings.json , never inside the code	One place to change, and the code stays clean.
All database traffic goes through AppDbContext	One single door to the DB, so nothing is hidden in random files.
After a code change: save, stop (Ctrl+C), then run again	The server reads the code only at startup. This is the usual reason for a 404.
bin/ , obj/ and secret files stay out of git	They are generated automatically and would only create conflicts.

6. Open items

Item	Details
Shared server name	appsettings.json contains localhost\SQLEXPRESS. If the teammate's SQL Server has a different name, the API will not connect on his machine. To be solved later with a personal settings file.
Files not created yet	Models (C# classes that mirror the tables), Controllers or more endpoints, and a Services layer for the login logic.